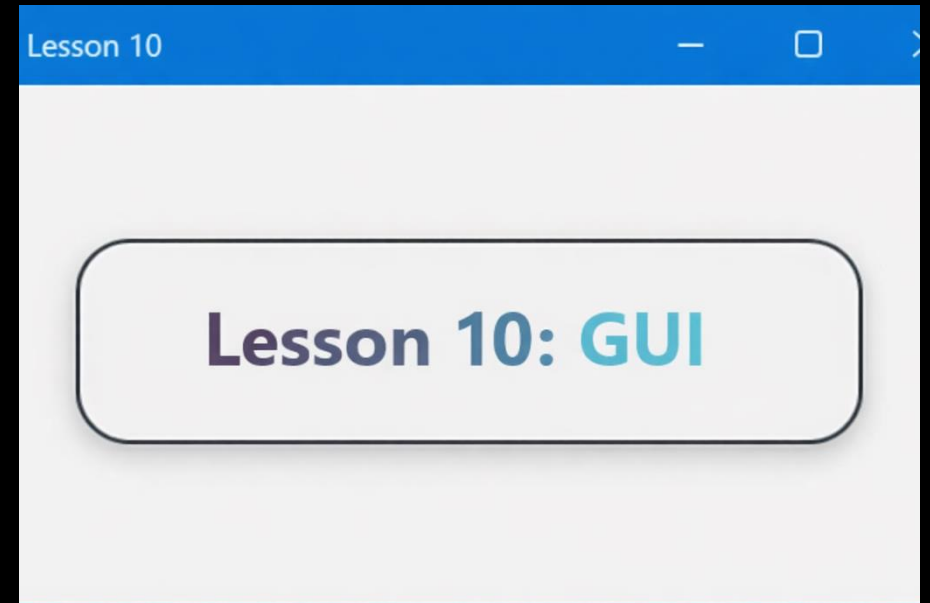


# OOP with Java

## Lesson 10: JavaFX GUI



your quiz:

- Open book: bring one page of notes (one sided; hand written)
- Focus on slides marked with "(QUIZ)" in their title

# JavaFX

- JavaFX is Java's modern GUI (Graphical User Interface) toolkit, designed to replace the older Swing library with more powerful, flexible tools for creating desktop applications. J
- JavaFX supports a range of rich media types (2D and 3D graphics, animations, video, and audio).
- Unlike Swing, JavaFX is entirely written in Java, uses CSS for styling, and supports FXML, an XML-based markup language to define the GUI structure.

# Why Use JavaFX for GUIs?

- JavaFX provides a cross-platform framework to develop GUI applications that can run on various operating systems without modification.
- Many companies use JavaFX for business and productivity tools, internal applications, and data visualization because it combines Java's robust backend capabilities with a sleek and powerful front end.

# Setting Up

- 1. Download the appropriate JavaFX version (e.g., JavaFX 21 for Java 21) from GluonHQ:  
<https://gluonhq.com/products/javafx>
- 2. Extract the JavaFX SDK directly into your project directory, naming it something like javafx-sdk; e.g. C:\javafx-sdk (or better yet put it in your project root folder)
- 3. IntelliJ IDEA
  - - File → Project Structure → Libraries
  - - Click the `+` button to add a new library and choose Java
  - - Select C:\javafx-sdk\lib
- 4. Configure VM Options
  - - Run → Edit Configurations → select your main class
  - - Modify options ... Add VM options ... `--module-path C:\javafx-sdk\lib --add-modules javafx.controls,javafx.fxml`
- 5. Create a "styles.css" file
  - Place any style files in a subfolder of your project (e.g., `./src/main/resources/styles.css`) and reference them in your code using `getClass().getResource("/styles.css")` to maintain portability.
- 6. Consider IDE Configuration
  - Make sure that your Project SDK and Project Language Level in IntelliJ align with your Java version, and update IntelliJ if needed. Students get free access to the Ultimate Edition, which may offer additional JavaFX support features.

# JavaFX vs. Swing

- Swing is the older GUI library in Java, and while it's still used in legacy applications, it lacks many modern features. Swing relies on its own rendering system, which doesn't support CSS or modern layouts, making it harder to create visually appealing apps by today's standards. In contrast, JavaFX supports CSS and FXML, offers more flexible layouts, and is designed with modern GUIs in mind.
- Example Comparison:
- Swing Button:
- `JButton button = new JButton("Click Me");`
- JavaFX Button:
- `Button button = new Button("Click Me");`
- JavaFX also offers better support for animations, 3D graphics, and multimedia.

# Four Main Concepts (QUIZ)

1. Stage is the (empty) window that holds a Scene. Without a Stage, there's no GUI
2. `launch(args)` is called in `main()`; it is required to start the JavaFX engine, and calls `sart()`
3. Scene is the content inside the window (aka the Stage). Without it, the window is blank; the Scene displays UI elements
4. `start()` is where we build everything; it sets up the whole GUI and defines what happens when the app starts

1. 

```
public void start(final Stage primaryStage){  
    primaryStage.show();  
}
```
2. 

```
public static void main(final String[] args){  
    launch(args);  
}
```
3. `primaryStage.setScene(scene);`
4. Override `start(final Stage primaryStage)`

# Structure of a JavaFX Application (QUIZ)

- A JavaFX application must **extend the Application class and override the start() method**, which is the entry point for your GUI code. Inside start(), you define the main Stage (the window) and set up the Scene (the content within the window).
- Example:
- `public class MyApp extends Application {   // MUST be public`
- `@Override`
- `public void start(final Stage primaryStage) {`
- `primaryStage.setTitle("Hello JavaFX!");`
- `primaryStage.show();`
- `}`
- `}`

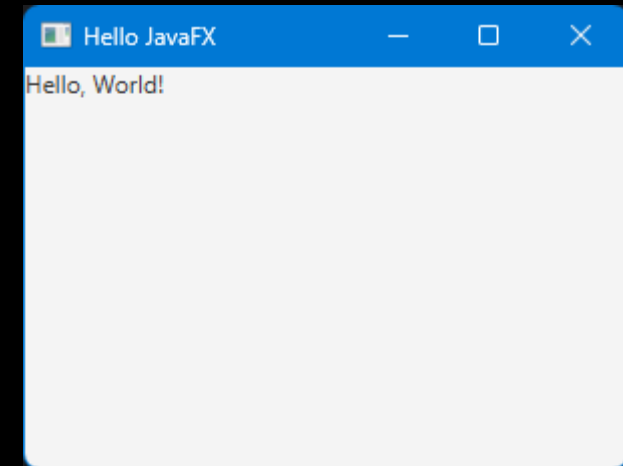
# Launching a JavaFX Application (QUIZ)

- In a JavaFX project, the main method is used only to call `Application.launch(args)`, which initializes the JavaFX environment and calls the `start()` method. This approach separates the application logic from the GUI creation.
- `public static void main(final String[] args) {`
- `Application.launch(args); // (or "launch(args)")`
- `}`



# JavaFX Hello World

- **public** class HelloWorld extends Application {
  - @Override
  - public void start(final Stage stage) {
    - final Label helloLabel = new Label("Hello, World!");
    - final VBox root = new VBox(helloLabel);
    - final Scene scene = new Scene(root, 300, 200);
    - 
    - stage.setScene(scene);
    - stage.setTitle("Hello JavaFX");
    - stage.show();
  - }
- public static void main(final String[] args) {
  - launch(args);
  - }
- }
- Best Practice: Always use a layout container (like VBox) to organize components.



# Main Method vs. Launching JavaFX

- JavaFX applications require `Application.launch(args)` to initialize the JavaFX runtime, not just the typical main method. Many students forget this step because traditional Java applications don't need it.
- `public static void main(final String[] args) {`
- `Application.launch(args); // Always required for JavaFX apps`
- `}`
- Failing to include `Application.launch()` will cause a runtime error because JavaFX needs its environment initialized through this call.

# Best Practices for JavaFX Application Structure

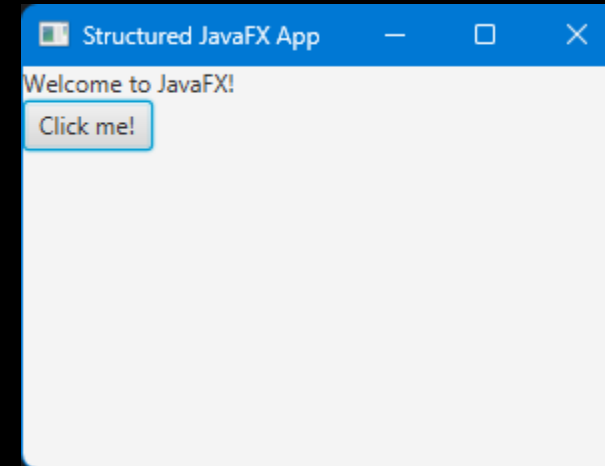
When organizing JavaFX applications, separate logic from the `start()` method to keep the code manageable and avoid long, unreadable methods.

Initialize complex components (like buttons or layouts) in separate methods and call them from `start()`.

`@Override`

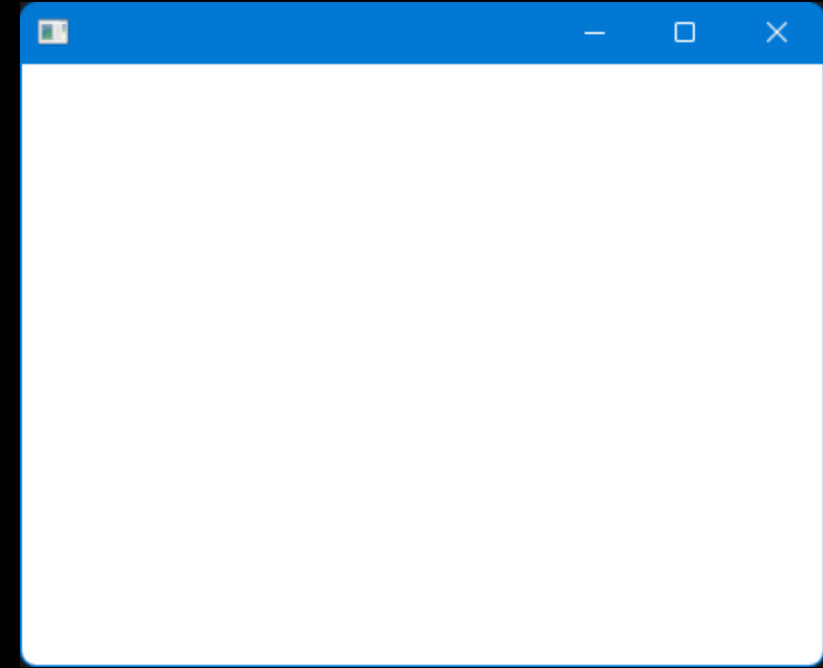
```
public void start(final Stage stage) {  
    VBox layout = createLayout();  
    Scene scene = new Scene(layout, 300, 200);  
    stage.setScene(scene);  
    stage.setTitle("Structured JavaFX App");  
    stage.show();  
}  
  
private VBox createLayout() {  
    Label label = new Label("Welcome to JavaFX!");  
    Button button = new Button("Click me!");  
    return new VBox(label, button);  
}
```

Best Practice: Use helper methods to organize layout components and keep `start()` clean and focused.



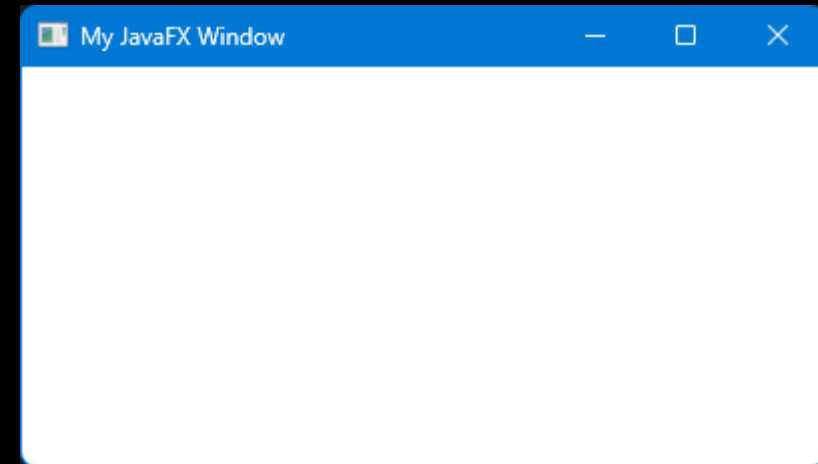
# Understanding Stage and Scene

- A Stage represents a top-level container or window.
  - The Scene is the content displayed within that Stage, containing nodes like buttons, text fields, and layouts.
  - Every JavaFX application has a primary Stage which is managed by the `start()` method.
  - The Scene organizes the visual components of the application in a hierarchical structure called the scene graph.
  - Nodes are arranged in the scene graph, and some nodes can contain child nodes.
- 
- `Stage stage = new Stage(); // Window`
  - `Scene scene = new Scene(new VBox(), 400, 300); // Content within the window`
  - `stage.setScene(scene);`
  - `stage.show();`



# Creating a Basic Window

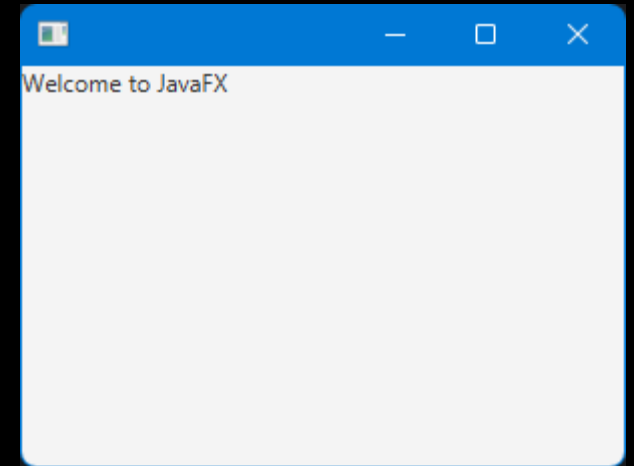
- To create a window in JavaFX, set the Stage title and add a Scene with a layout. A Stage can display only one Scene at a time, but you can replace the current scene with a new one if needed.
- ```
public class BasicWindow extends Application {  
    @Override  
    public void start(final Stage primaryStage) {  
        primaryStage.setTitle("My JavaFX Window");  
  
        VBox layout = new VBox();  
        Scene scene = new Scene(layout, 400, 200);  
  
        primaryStage.setScene(scene);  
        primaryStage.show();  
    }  
}
```



Always call `show()` on the Stage to make the window visible; otherwise, it won't display.

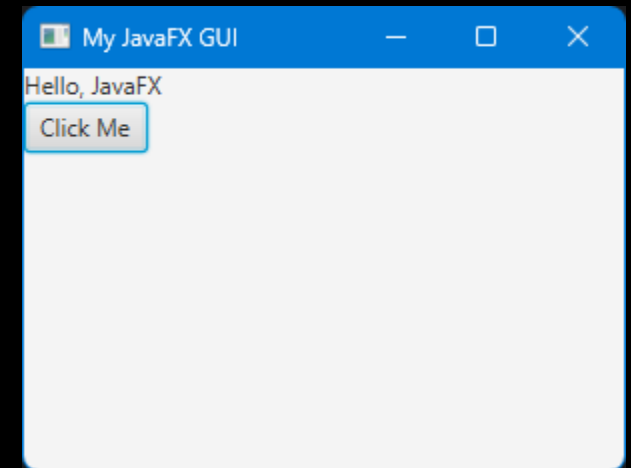
# Adding ("Setting") a Scene to a Stage

- In JavaFX, a Scene is added to the Stage by calling `setScene()`.
  - A Scene defines the size and content structure for your application's window.
  - You can later change the scene on the same stage, making it flexible for multi-screen applications.
- 
- `VBox layout = new VBox(new Label("Welcome to JavaFX"));`
  - `Scene scene = new Scene(layout, 300, 200);`
  - `primaryStage.setScene(scene);`
  - `primaryStage.show();`
- 
- Define Scene content and layout first, then add it to the Stage using `setScene()` for better organization.



# Using Scene Graphs (QUIZ)

- JavaFX organizes components into a hierarchical scene graph where each component is a "node."
- Many nodes can contain other nodes as children, forming a tree-like structure.
- The root node (e.g., a layout container) is at the top, and other nodes branch out under it.
- `VBox root = new VBox();`
- `Button btn = new Button("Click Me");`
- `Label label = new Label("Hello, JavaFX");`
- `root.getChildren().addAll(label, btn);`
- `Scene scene = new Scene(root, 300, 200);`
- `// Set the scene on the stage`
- `primaryStage.setScene(scene);`
- `// Set a title for the window (optional)`
- `primaryStage.setTitle("My JavaFX GUI");`
- `// Show the stage`
- `primaryStage.show();`
- Start with a single root node (e.g., `VBox`, `HBox`) to manage child nodes efficiently.



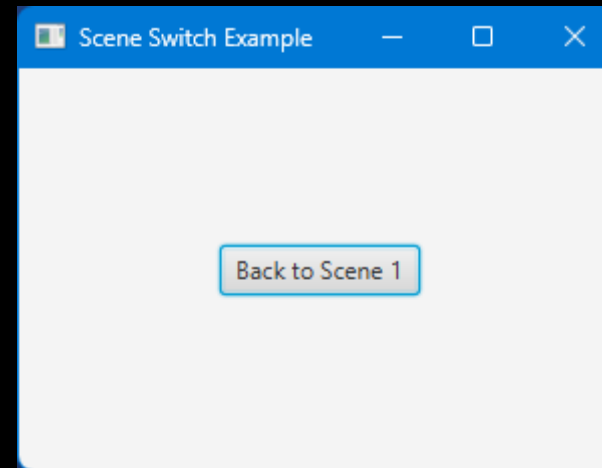
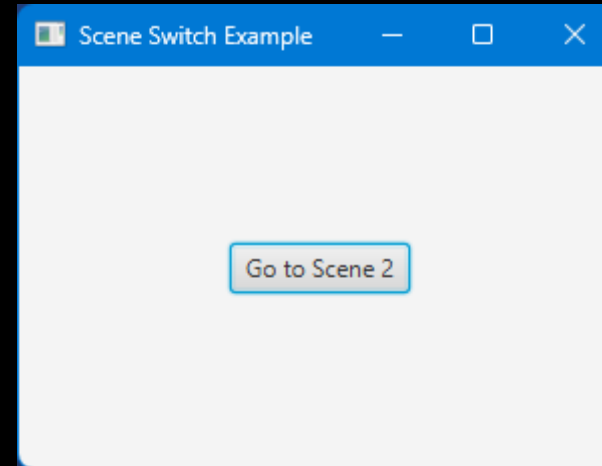
# Stage is the Top-Level Container

- Only a Stage can contain a Scene; don't try to add a scene or component directly to another scene.
- Only one Scene can be displayed on a Stage at a time, so to swap screens or views, set a new Scene on the same Stage.
- `primaryStage.setScene(scene1); // Display scene1`
- `primaryStage.setScene(scene2); // Replace with scene2`
- Example on next slide...



# Switching Scenes

- `@Override`
- `public void start(final Stage primaryStage) {`
- `// Create first scene with a button to switch to the second scene`
- `Button button1 = new Button("Go to Scene 2");`
- `StackPane layout1 = new StackPane(button1);`
- `Scene scene1 = new Scene(layout1, 300, 200);`
- `// Create second scene with a button to switch back to the first scene`
- `Button button2 = new Button("Back to Scene 1");`
- `StackPane layout2 = new StackPane(button2);`
- `Scene scene2 = new Scene(layout2, 300, 200);`
- `// Set actions for buttons to switch scenes`
- `button1.setOnAction(e -> primaryStage.setScene(scene2));`
- `button2.setOnAction(e -> primaryStage.setScene(scene1));`
- `// Set initial scene and show stage`
- `primaryStage.setScene(scene1);`
- `primaryStage.setTitle("Scene Switch Example");`
- `primaryStage.show();`
- `}`

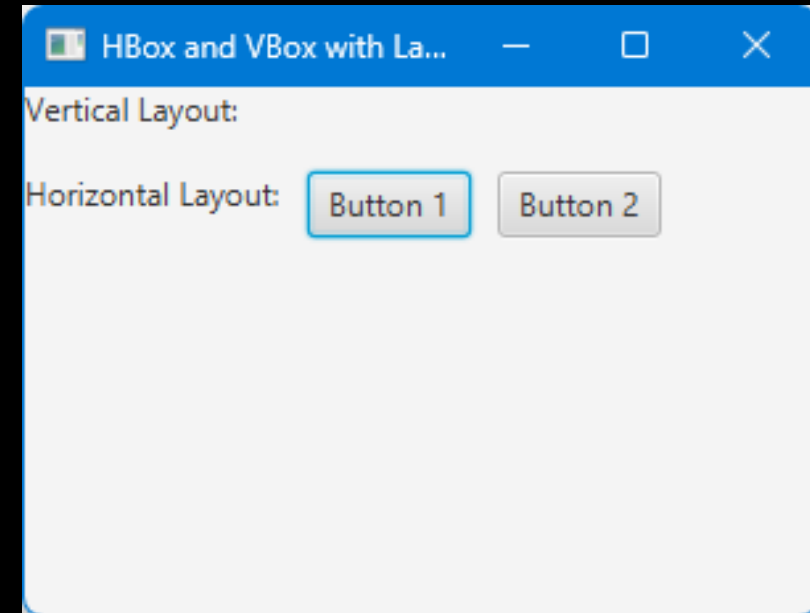


# Introduction to Layouts

- Layouts in JavaFX help arrange UI components.
- Common layouts include VBox (vertical), HBox (horizontal), and GridPane (tabular).
- Layouts help to avoid absolute positioning and provide a responsive design.
- Best Practice: Use layouts instead of manually positioning elements to ensure that your UI adapts to various screen sizes.
- See example next slide...

# Vertical and Horizontal Layouts

- `@Override`
- `public void start(final Stage primaryStage) {`
- `// Create a vertical label at the top`
- `Label verticalLabel = new Label("Vertical Layout:");`
- `// Create a horizontal label inside the HBox`
- `Label horizontalLabel = new Label("Horizontal Layout:");`
- `// Create an HBox with a label and two buttons, arranged horizontally`
- `HBox hbox = new HBox(10); // 10 pixels spacing between buttons`
- `Button button1 = new Button("Button 1");`
- `Button button2 = new Button("Button 2");`
- `hbox.getChildren().addAll(horizontalLabel, button1, button2);`
- `// Create a VBox to arrange the vertical label and HBox vertically`
- `VBox vbox = new VBox(15); // 15 pixels spacing between elements`
- `vbox.getChildren().addAll(verticalLabel, hbox);`
- `// Set up the scene with the VBox as the root layout`
- `Scene scene = new Scene(vbox, 300, 200);`
- `// Configure the stage`
- `primaryStage.setScene(scene);`
- `primaryStage.setTitle("HBox and VBox with Labels");`
- `primaryStage.show();`
- `}`



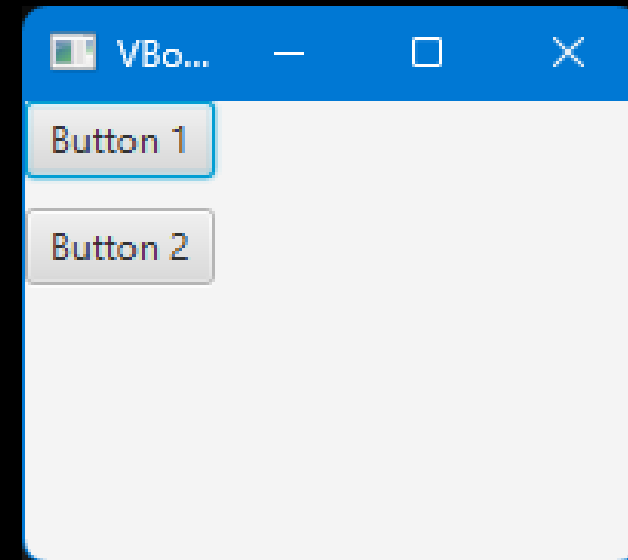
# Using VBox Layout

- The VBox layout arranges nodes vertically from top to bottom.
  - Simple and effective for stacking components.
  - Spacing and padding can be added to customize the look.
- ```
// Create a VBox layout with 10 pixels spacing
VBox vbox = new VBox(10);

// Add buttons to the VBox
Button button1 = new Button("Button 1");
Button button2 = new Button("Button 2");
vbox.getChildren().addAll(button1, button2);

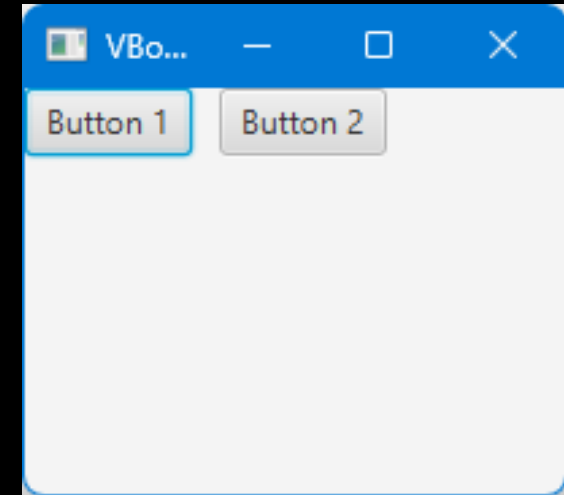
// Create a Scene with the VBox as the root node
Scene scene = new Scene(vbox, 200, 150);

// Set the scene on the primary stage and display it
primaryStage.setScene(scene);
primaryStage.setTitle("VBox Example");
primaryStage.show();
```
- Avoid adding too many elements to VBox without spacing, as it can appear cluttered.



# Using HBox Layout

- The HBox layout arranges nodes horizontally, side-by-side.
- Use HBox for aligning elements in a row, like a toolbar or navigation bar.
- `HBox hbox = new HBox(15); // 15 pixels spacing`
- `Button btn1 = new Button("Home");`
- `Button btn2 = new Button("Settings");`
- `hbox.getChildren().addAll(btn1, btn2);`
- `Scene scene = new Scene(hbox, 300, 100);`
- `// Set the scene on the primary stage and display it`
- `primaryStage.setScene(scene);`
- `primaryStage.setTitle("VBox Example");`
- `primaryStage.show();`
- Adjust spacing between elements for better readability and user experience.

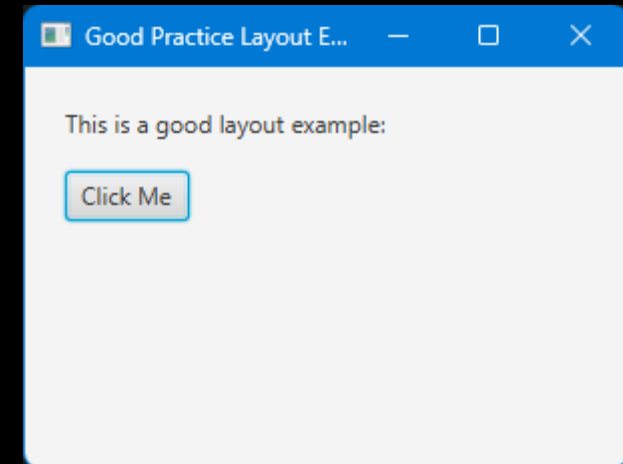


# Do Not Hardcode Positions

- Hardcoding positions, like setting `setLayoutX` and `setLayoutY`, can cause issues with resizing and responsiveness.
- Instead, use layout containers (e.g., `VBox`, `HBox`, `GridPane`) to control component placement.
- `Button btn = new Button("Click Me");`
- `btn.setLayoutX(100);` // Avoid hardcoded positioning
- `btn.setLayoutY(50);`
- Always use layout managers for scalable design.
- See next slide for examples...

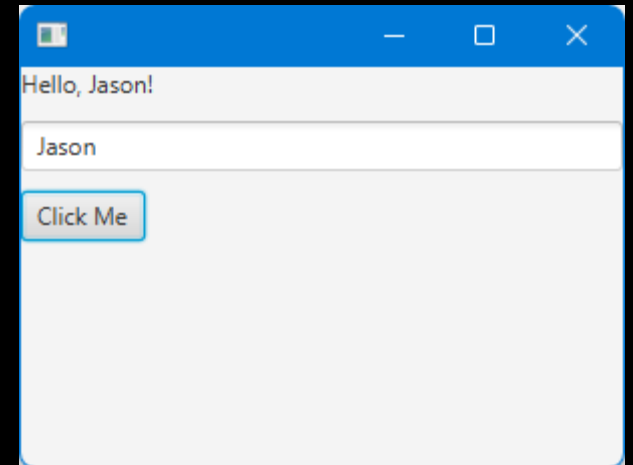
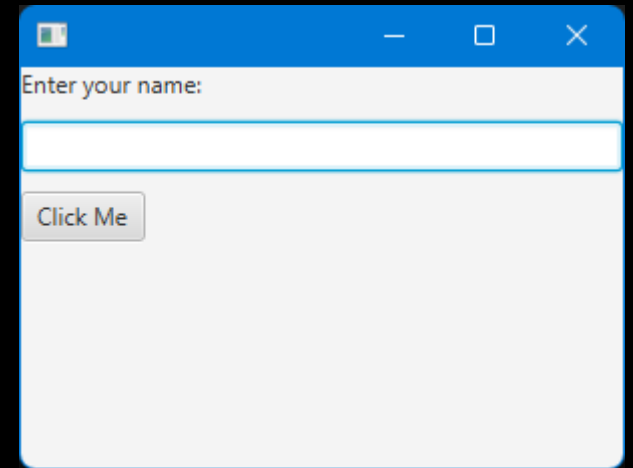
# Good Component Placement

- `public class Gui extends Application {`
- `@Override`
- `public void start(final Stage primaryStage) {`
- `Label label = new Label("This is a good layout example:");`
- `Button btn = new Button("Click Me");`
- `// Create an HBox to hold the button and add spacing`
- `HBox hbox = new HBox(10); // 10 pixels spacing`
- `hbox.getChildren().add(btn);`
- `// Create a VBox to stack the label and the HBox vertically`
- `VBox vbox = new VBox(15); // 15 pixels spacing between elements`
- `vbox.getChildren().addAll(label, hbox);`
- `// Set padding and alignment in the VBox (optional, for better appearance)`
- `vbox.setStyle("-fx-padding: 20;");`
- `// Create the Scene with the VBox as the root layout`
- `Scene scene = new Scene(vbox, 300, 200);`
- `// Configure the Stage`
- `primaryStage.setScene(scene);`
- `primaryStage.setTitle("Good Practice Layout");`
- `primaryStage.show();`
- `}`
- `}`



# JavaFX Controls Overview (QUIZ)

- JavaFX provides a variety of controls (UI components) to interact with users, including buttons, labels, text fields, and more complex controls like combo boxes and tables.
- Controls are essential for building interactive applications, enabling users to input data and receive feedback.
- Button: Used to trigger actions.
- Label: Displays text information.
- TextField: Allows user input of single-line text.
- Each control is a node that can be added to layout containers like VBox or HBox.
- **Label greeting = new Label("Enter your name:");**
- **TextField textField = new TextField();**
- **Button button = new Button("Click Me");**
- **button.setOnAction(e -> greeting.setText("Hello, " + textField.getText() + "!!"));**
- **stage.setScene(new Scene(new VBox(10, greeting, textField, button), 300, 200));**
- **stage.show();**





# Creating a Button

- `Button button = new Button("Submit");`
- `VBox layout = new VBox(button);`
- `Scene scene = new Scene(layout, 200, 100);`
- Avoid assigning long labels to buttons as it may affect readability and cause alignment issues.

# Using Labels for Display Text

- Labels in JavaFX are used to display text or information.
- Labels are non-interactive (read-only); users cannot modify their text directly.
- `Label greeting = new Label("Welcome to JavaFX!");`
- `VBox layout = new VBox(greeting);`
- `Scene scene = new Scene(layout, 300, 150);`
- Use labels to display instructions or status updates and avoid using them for data that requires user input.

# Text Input with TextField

- TextField is a JavaFX control that allows users to input single-line text.
- You can set initial text with `setText()` and retrieve user input using `getText()`.
- TextField is typically used for short text inputs like names or search queries.
- `TextField nameField = new TextField();`
- `nameField.setPromptText("Enter your name"); // Placeholder text`
- `String userName = nameField.getText(); // Retrieve input`
- Avoid using TextField for long text input; use TextArea instead.

# TextArea

- A JavaFX control designed for multi-line text input, making it ideal for longer inputs such as descriptions, comments, or messages.
- - Setting Initial Text: Use `setText()` to provide default or initial text content.
- - Getting User Input: Retrieve the user's input using `getText()`.
- TextArea supports line wrapping and allows users to type multiple lines, which makes it more suitable than TextField for longer text.
- ```
TextArea commentsArea = new TextArea();
```
- ```
commentsArea.setPromptText("Enter your comments");
```

 // Placeholder text
- ```
String comments = commentsArea.getText();
```

 // Retrieve input
- - Example Use Cases: Feedback forms, notes, chat messages, etc.
- Unlike TextField, TextArea supports both horizontal and vertical scroll bars, which appear *automatically* when the content exceeds the visible area.

# Keep Controls in Layout Containers

- All controls, like buttons, labels, and text fields, must be added to layout containers (e.g., VBox, HBox, GridPane).
- JavaFX does not allow adding controls directly to the Scene; they must first be wrapped in a container.
- Incorrect:
  - `Scene scene = new Scene(button); // Won't work`
- Correct:
  - `VBox layout = new VBox(button);`
  - `Scene scene = new Scene(layout, 200, 100);`
- Forgetting to use a layout container will result in a compile-time error.

# Introduction to Events in JavaFX

- JavaFX is event driven, meaning components respond to user actions through events.
- Each control can listen for events like clicks or text changes.
- Event handling enables user interaction and is essential for creating responsive GUIs.
- `Button button = new Button("Click Me");`
- `button.setOnAction(event -> System.out.println("Button clicked!"));`
- Use lambdas for concise event handling, especially for simple actions.

# Setting an Action on a Button

- Button clicks are one of the most common events in GUIs.
- To handle a button click, use `setOnAction()` with an `EventHandler`.
- When the button is clicked, the code inside the handler executes.
- `Button submitButton = new Button("Submit");`
- `submitButton.setOnAction(event -> System.out.println("Submitted!"));`
- Always ensure the action code is concise. Long actions should be extracted to separate methods for readability.

# Event Handling: Lambda Expressions (QUIZ)

- Lambda expressions offer a compact way to handle events, making code more readable.
- Instead of writing verbose inner classes, lambdas allow you to write short, single-line event handlers.
- Lambdas are particularly useful when the handler is simple and doesn't require additional context.
- `Button saveButton = new Button("Save");`
- `saveButton.setOnAction(event -> System.out.println("Data saved"));`
- Use lambdas for straightforward events, but consider method references or separate methods for more complex logic.



# Avoid Long Event Code in Lambdas

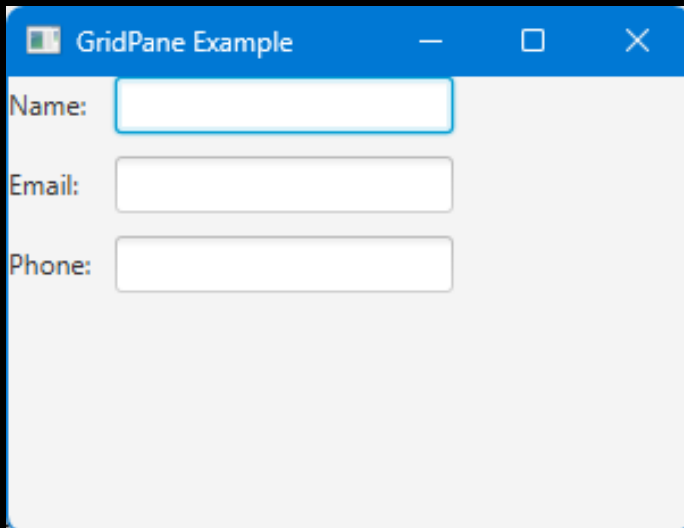
- For complex actions, avoid writing long lambda expressions within `setOnAction()` calls. Long lambdas reduce readability and can clutter the code. Instead, use separate methods to handle complex logic, keeping the main flow of code clean and readable.

- ```
// Button with a too-long lambda expression
```
- ```
Button longLambdaButton = new Button("Calculate (Long Lambda)");
```
- ```
longLambdaButton.setOnAction(event -> {
```
- ```
    int result = 0;
```
- ```
    for (int i = 1; i <= 10; i++) {
```
- ```
        result += i * i;
```
- ```
    }
```
- ```
    System.out.println("Long lambda calculation complete: " + result);
```
- ```
});
```

- ```
// Button with a refactored handler using a separate method
```
- ```
Button refactoredButton = new Button("Calculate (Refactored)");
```
- ```
refactoredButton.setOnAction(event -> performCalculations());
```
- ```
// Layout and scene setup
```
- ```
VBox vbox = new VBox(10, longLambdaButton, refactoredButton);
```
- ```
Scene scene = new Scene(vbox, 300, 200);
```
- ```
primaryStage.setScene(scene);
```
- ```
primaryStage.setTitle("Complex Action Example");
```
- ```
primaryStage.show();
```
- ```
}
```
- ```
// Separate method to handle calculations
```
- ```
private void performCalculations() {
```
- ```
    int result = 0;
```
- ```
    for (int i = 1; i <= 10; i++) {
```
- ```
        result += i * i;
```
- ```
    }
```
- ```
    System.out.println("Refactored calculation complete: " + result);
```
- ```
}
```

# GridPane for Table Layouts

- GridPane is a JavaFX layout that arranges components in rows and columns, similar to a table structure.
- Each component is placed within a specific cell, making it ideal for creating form-like layouts or aligning components in a grid.
- You can specify the row and column for each component, as well as set spacing and padding to control the layout.
- Use GridPane for layouts that require structured alignment of elements, like forms or tables.



```
// GridPane layout with gaps between cells
GridPane grid = new GridPane();
grid.setHgap(10); // Horizontal gap between cells
grid.setVgap(10); // Vertical gap between cells
```

```
// Add components to the grid
Label nameLabel = new Label("Name:");
TextField nameField = new TextField();
grid.add(nameLabel, 0, 0); // Column 0, Row 0
grid.add(nameField, 1, 0); // Column 1, Row 0
```

```
Label emailLabel = new Label("Email:");
TextField emailField = new TextField();
grid.add(emailLabel, 0, 1); // Column 0, Row 1
grid.add(emailField, 1, 1); // Column 1, Row 1
```

```
Label phoneLabel = new Label("Phone:");
TextField phoneField = new TextField();
grid.add(phoneLabel, 0, 2); // Column 0, Row 2
grid.add(phoneField, 1, 2); // Column 1, Row 2
```

```
// Create and set up the scene
Scene scene = new Scene(grid, 300, 200);
primaryStage.setScene(scene);
primaryStage.setTitle("GridPane Example");
primaryStage.show();
```

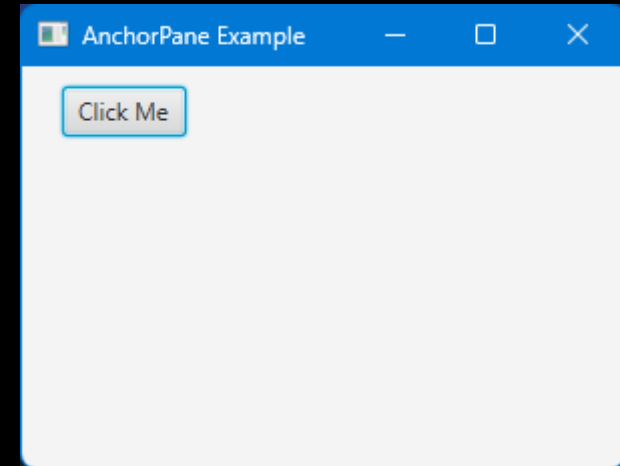
# AnchorPane for Flexible Layouts

- AnchorPane allows nodes to be anchored to the edges of the pane.
  - This layout type is particularly useful when you want to control the positioning of nodes in relation to the pane's borders.
  - By setting anchor constraints, you can make components dynamically adjust to the pane's resizing.
- ```
AnchorPane anchorPane = new AnchorPane();

Button button = new Button("Click Me");
AnchorPane.setTopAnchor(button, 10.0); // 10 pixels from the top
AnchorPane.setLeftAnchor(button, 20.0); // 20 pixels from the left

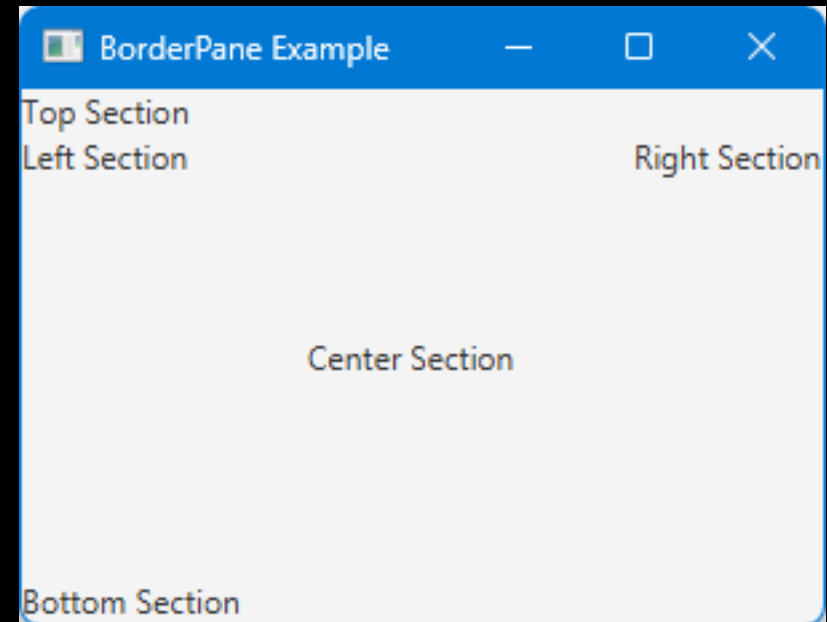
anchorPane.getChildren().add(button);

Scene scene = new Scene(anchorPane, 300, 200);
primaryStage.setScene(scene);
primaryStage.setTitle("AnchorPane Example");
primaryStage.show();
```
- Overuse of AnchorPane can lead to rigid designs; use it only when specific positioning is necessary.



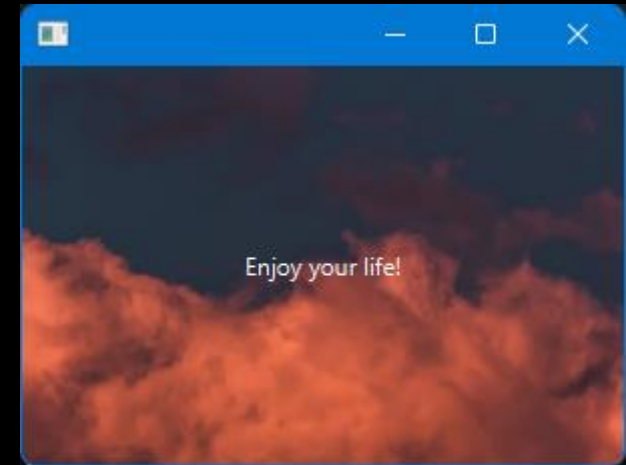
# BorderPane Layout

- BorderPane divides the layout into five regions: Top, Bottom, Left, Right, and Center.
- Each region can hold a single node, making BorderPane suitable for layouts that need clear sections, like a typical web page structure.
- ```
BorderPane borderPane = new BorderPane();
```
- ```
borderPane.setTop(new Label("Top Section"));
```
- ```
borderPane.setLeft(new Label("Left Section"));
```
- ```
borderPane.setCenter(new Label("Center Section"));
```
- ```
borderPane.setRight(new Label("Right Section"));
```
- ```
borderPane.setBottom(new Label("Bottom Section"));
```
- ```
Scene scene = new Scene(borderPane, 300, 200);
```
- ```
primaryStage.setScene(scene);
```
- ```
primaryStage.setTitle("BorderPane Example");
```
- ```
primaryStage.show();
```
- Best Practice: Use BorderPane for layouts that need a fixed structure, like a header, footer, and central content area.



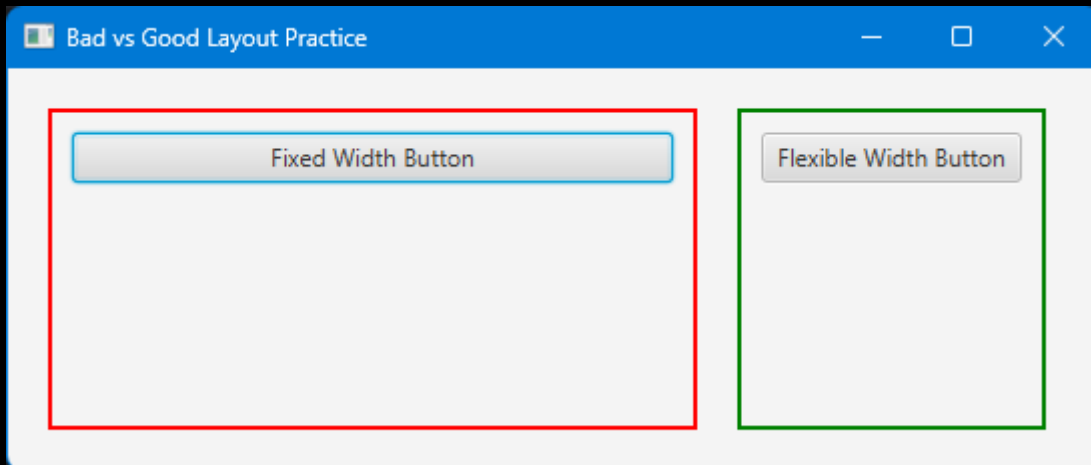
# StackPane for Overlaying Nodes

- StackPane allows nodes to be stacked on top of each other, with the first added node at the bottom and the last on top.
- This layout is ideal for layering components, such as displaying text over an image.
- @Override
- public void start(Stage stage) {
- ImageView imageView = new ImageView(new Image("file:sky.jpg"));
- Label overlayText = new Label("Enjoy your life!");
- overlayText.setStyle("-fx-text-fill: white;");
- StackPane stackPane = new StackPane(imageView, overlayText);
- stage.setScene(new Scene(stackPane, 300, 200));
- stage.show();
- }
- Be cautious with transparency in layered elements to avoid visibility issues.



# Test Layouts for Different Screen Sizes

- Gotcha: JavaFX layouts adapt to screen size, but some designs may not appear correctly on smaller or larger displays.
- Always test your layout on various screen sizes to ensure readability and usability.
- Avoid hardcoding dimensions, and use layout containers to create flexible designs.
- Best Practice: Avoid fixed dimensions whenever possible. Let layouts adjust automatically for optimal display on various screen sizes.



```
// Bad Practice: Fixed Width Button
Button fixedWidthButton = new Button("Fixed Width Button");
fixedWidthButton.setPrefWidth(300); // may not adapt well on smaller screens
```

```
// Good Practice: Flexible Width Button
Button flexibleWidthButton = new Button("Flexible Width Button");
flexibleWidthButton.setMaxWidth(Double.MAX_VALUE); // adapt to screen size
```

```
// Layouts
VBox badPracticeBox = new VBox(10, fixedWidthButton);
badPracticeBox.setPadding(new Insets(10));
badPracticeBox.setStyle("-fx-border-color: red; -fx-border-width: 2; -fx-padding: 10;");
```

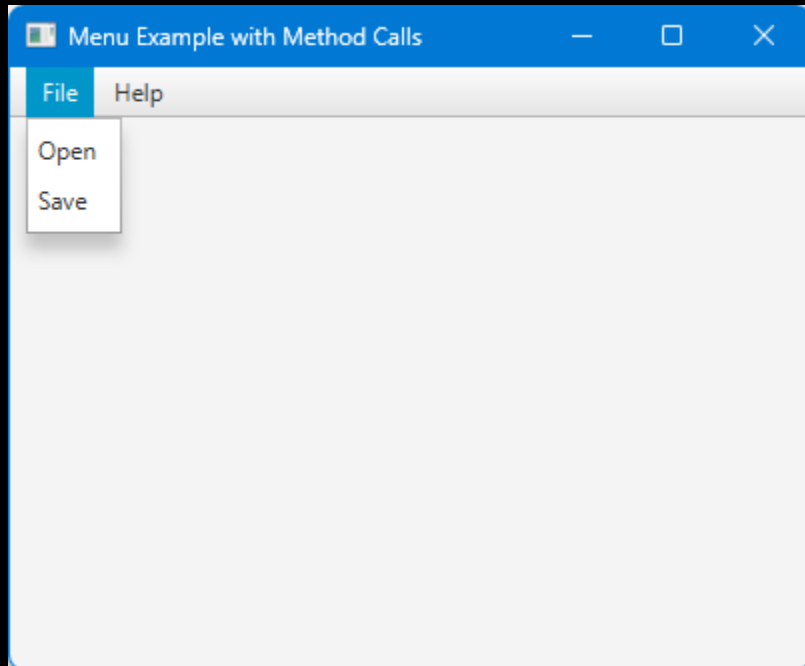
```
VBox goodPracticeBox = new VBox(10, flexibleWidthButton);
goodPracticeBox.setPadding(new Insets(10));
goodPracticeBox.setStyle("-fx-border-color: green; -fx-border-width: 2; -fx-padding: 10;");
```

```
// Add both to HBox for side-by-side comparison
HBox root = new HBox(20, badPracticeBox, goodPracticeBox);
root.setPadding(new Insets(20));
```

```
Scene scene = new Scene(root, 600, 200);
stage.setScene(scene);
stage.setTitle("Bad vs Good Layout Practice");
stage.show();
```

# Menus and Callbacks

- Scenes have panes (e.g. to place items at the top of a window)
- Panes have MenuBars
- MenuBars have Menus
- Menus have MenuItems
- MenuItems have "setOnAction" action handlers (methods to run)



```
public class MenuExample extends Application {
    @Override
    public void start(final Stage stage) {
        MenuBar menuBar = new MenuBar();
        Menu fileMenu = new Menu("File");
        Menu helpMenu = new Menu("Help");

        MenuItem openItem = new MenuItem("Open");
        openItem.setOnAction(e -> openFile());
        MenuItem saveItem = new MenuItem("Save");
        saveItem.setOnAction(e -> saveFile());
        MenuItem aboutItem = new MenuItem("About");
        aboutItem.setOnAction(e -> showAbout());

        fileMenu.getItems().addAll(openItem, saveItem);
        helpMenu.getItems().add(aboutItem);

        menuBar.getMenus().addAll(fileMenu, helpMenu);

        BorderPane root = new BorderPane();
        root.setTop(menuBar);

        Scene scene = new Scene(root, 400, 300);
        stage.setScene(scene);
        stage.setTitle("Menu Example with Method Calls");
        stage.show();
    }

    private void openFile() {
        System.out.println("Open File method called!");
    }
    private void saveFile() {
        System.out.println("Save File method called!");
    }
    private void showAbout() {
        System.out.println("About method called!");
    }
}
```

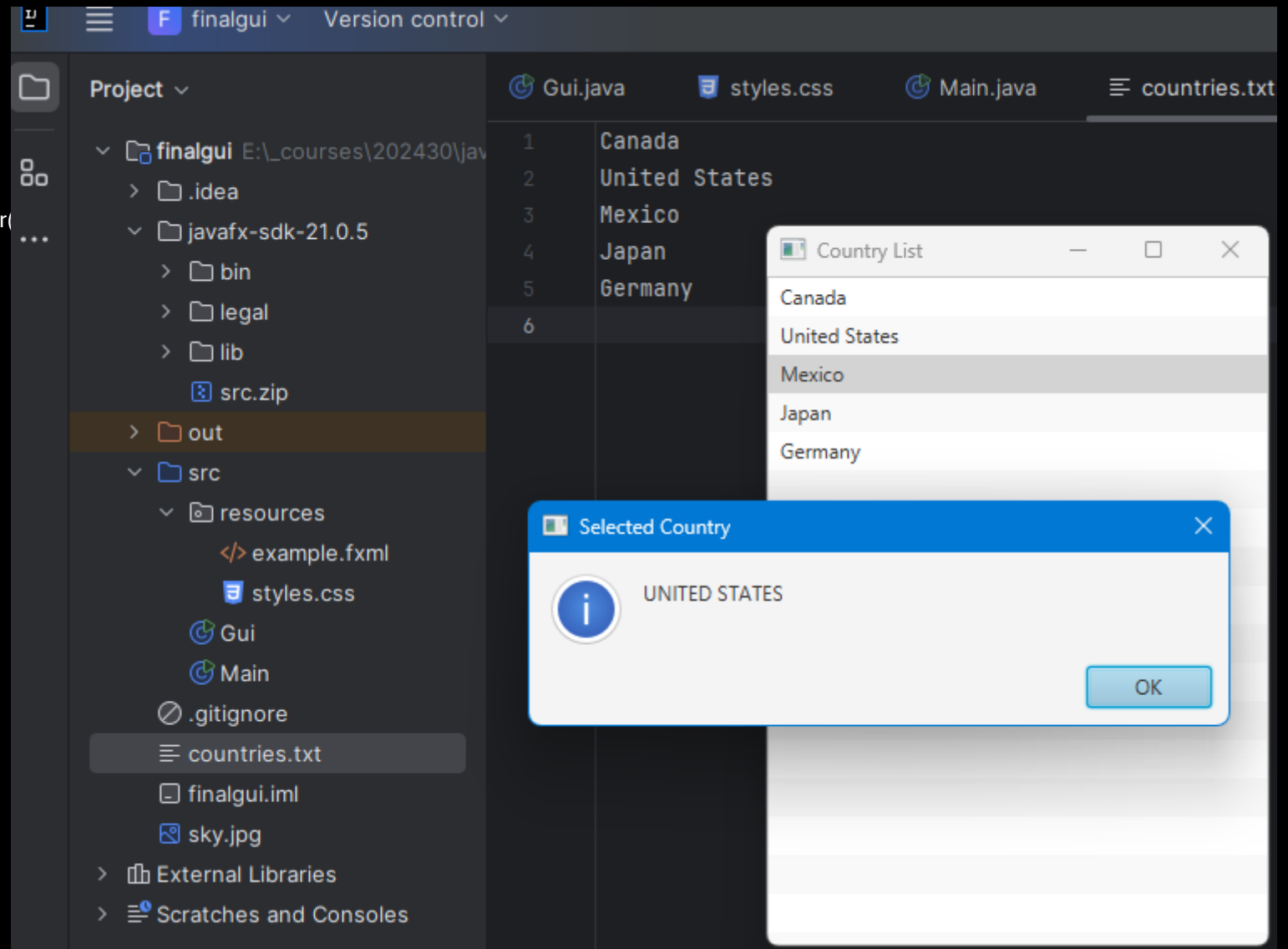
# Listview

```
@Override
public void start(final Stage primaryStage) {
    List<String> countries;
    try {
        countries = Files.readAllLines(Path.of("countries.txt"));
    } catch (final IOException e) {
        showErrorDialog("Error reading file: " + e.getMessage());
        return;
    }
    ListView<String> listView = new ListView<>();
    listView.getItems().addAll(countries);

    listView.getSelectionModel().selectedItemProperty().addListener(
        (observable, oldValue, newValue) -> {
            if (newValue != null) {
                showInfoDialog(newValue.toUpperCase());
            }
        });
    VBox root = new VBox(listView);
    primaryStage.setScene(new Scene(root, 300, 400));
    primaryStage.setTitle("Country List");
    primaryStage.show();
}

private void showInfoDialog(final String message) {
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setTitle("Selected Country");
    alert.setHeaderText(null);
    alert.setContentText(message);
    alert.showAndWait();
}

private void showErrorDialog(final String message) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("Error");
    alert.setHeaderText(null);
    alert.setContentText(message);
    alert.showAndWait();
}
}
```





# Threading and GUIs

- JavaFX requires some understanding of threading, especially for tasks that might take time, like fetching data from a database, making network requests, or performing complex calculations.
- JavaFX, like Swing, has a single-threaded UI model where all UI changes must happen on the **JavaFX Application Thread**.
- Running long tasks on this thread can freeze the UI, resulting in a poor user experience.
- Following is a breakdown of how threading is handled in JavaFX and what we must know.

# The JavaFX Application Thread (QUIZ)

- JavaFX has a designated main thread for all UI updates, called the **JavaFX Application Thread**.
- Any code that changes the UI (like updating labels, moving elements, etc.) must run on this thread.
- Warning:
- If you run time-consuming operations on this thread, it will block, making your UI unresponsive. So, it's essential to handle long-running tasks on a separate thread.

# JavaFX's Solution for Background Tasks (QUIZ)

- JavaFX provides classes specifically designed to help run tasks off the UI thread and then safely update the UI once they complete:
- Task<V> and Service<V>: These classes are part of the javafx.concurrent package and **make it easier to run background tasks and update the UI when they finish.**
- 1. Task<V> represents a single background operation, useful for one-time tasks.
- 2. Service<V> is suitable for tasks that need to be run multiple times or be restarted.
- These classes support properties like onSuccessed, onFailed, onCancelled, which let you specify actions to take when the task completes, fails, or is canceled, making it easier to handle UI updates based on task completion.

# Task<V>

The V represents the return type of the task's computation.

It is a generic type parameter, so the type is specified when creating a Task to define the type of result the task will produce once it completes.

- In particular, V is the type of the result returned by the call() method of the Task.
- When the Task finishes successfully, the result can be retrieved using task.getValue(), and the type of this value is V.

Example 1: returning a String Result

```
Task<String> stringTask = new Task<>() {  
    @Override  
    protected String call() {  
        return "Hello, World!";  
    }  
};
```

- The V is specified as String, so the task returns a String when completed.
- Call stringTask.getValue() to retrieve the "Hello, World!" result.

Example 2: returning an Integer Result

```
Task<Integer> intTask = new Task<>() {  
    @Override  
    protected Integer call() {  
        return 42;  
    }  
};
```

- Example 3: using Void for No Result
- Task<Void> voidTask = new Task<>() {
  - @Override
  - protected Void call() {
    - System.out.println("Doing some work...");
    - return null; // Void requires you to return null
  - }
- };
- - The V is Void, meaning the task does not produce a result.
- - Typically use this when the task performs actions (e.g., writing to a file) but doesn't return a value.

Now the GUI will not block even if this task takes a long time.

- Full example on the next slide...

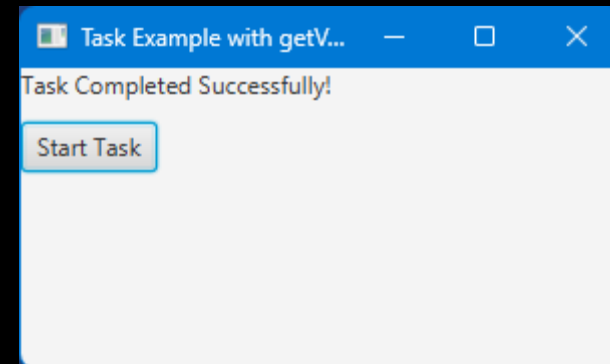
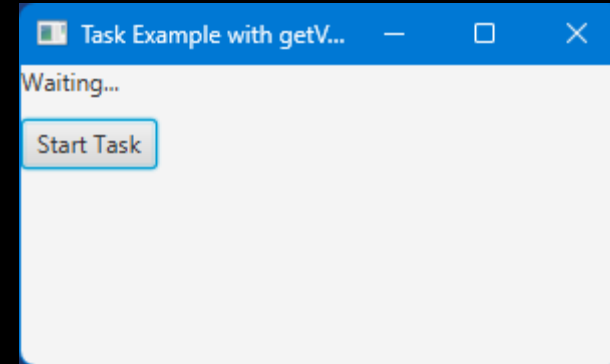
# Task<V>

```
public class Gui extends Application {
    @Override
    public void start(final Stage stage) {
        Label statusLabel = new Label("Waiting...");
        Button startButton = new Button("Start Task");
        // Define a Task that returns a String result
        Task<String> task = new Task<>() {
            @Override
            protected String call() throws InterruptedException {
                Thread.sleep(2000); // Simulate background work; 2000ms
                return "Task Completed Successfully!";
            }
        };

        // Update the label when the task succeeds
        task.setOnSucceeded(e -> {
            String result = task.getValue(); // Retrieve the result of the task
            statusLabel.setText(result); // Update the label with the result
        });

        // Start the task on a new thread when the button is clicked
        startButton.setOnAction(e -> new Thread(task).start());

        VBox root = new VBox(10, statusLabel, startButton);
        stage.setScene(new Scene(root, 300, 150));
        stage.setTitle("Task Example with getValue()");
        stage.show();
    }
    public static void main(final String[] args) {
        launch(args);
    }
}
```



# Service<V>

```
public class Gui extends Application {
    @Override
    public void start(final Stage stage) {
        Label statusLabel = new Label("Waiting...");
        Button controlButton = new Button("Start");

        // Define a Service for repeated background work
        Service<String> service = new Service<>() {
            @Override
            protected Task<String> createTask() {
                return new Task<>() {
                    @Override
                    protected String call() throws InterruptedException {
                        if (Math.random() < 0.3) throw new RuntimeException("Random failure!");
                        Thread.sleep(2000); // Simulate background work; 2000 ms
                        return "Success!";
                    }
                };
            }
        };

        // Define UI updates for each service state
        service.setOnRunning(e -> {
            statusLabel.setText("Running...");
            controlButton.setText("Cancel");
        });
        service.setOnSucceeded(e -> {
            statusLabel.setText("Completed: " + service.getValue());
            controlButton.setText("Restart");
        });
    }
}
```

```
        service.setOnFailed(e -> {
            statusLabel.setText("Failed: " + service.getException().getMessage());
            controlButton.setText("Restart");
        });
        service.setOnCancelled(e -> {
            statusLabel.setText("Cancelled!");
            controlButton.setText("Restart");
        });

        // Handle button click to cancel or restart the service
        controlButton.setOnAction(e -> {
            if (service.isRunning()) {
                service.cancel(); // Cancel the running service
            } else {
                service.restart(); // Restart the service
            }
        });

        VBox root = new VBox(10, statusLabel, controlButton);
        stage.setScene(new Scene(root, 300, 150));
        stage.setTitle("Service Example with Dynamic Button");
        stage.show();
    }

    public static void main(final String[] args) {
        launch(args);
    }
}
```

This example simulates a failure, and lets us cancel and/or repeat service runs.

# Using Platform.runLater() for UI Updates

- To update the UI from a background thread, use `Platform.runLater()`.
- This method schedules a runnable to be executed on the JavaFX Application Thread, allowing safe updates to UI components.
- `Platform.runLater(() -> {`
- `// Code to update UI components safely`
- `label.setText("Task completed!");`
- `});`
- See example next slide...

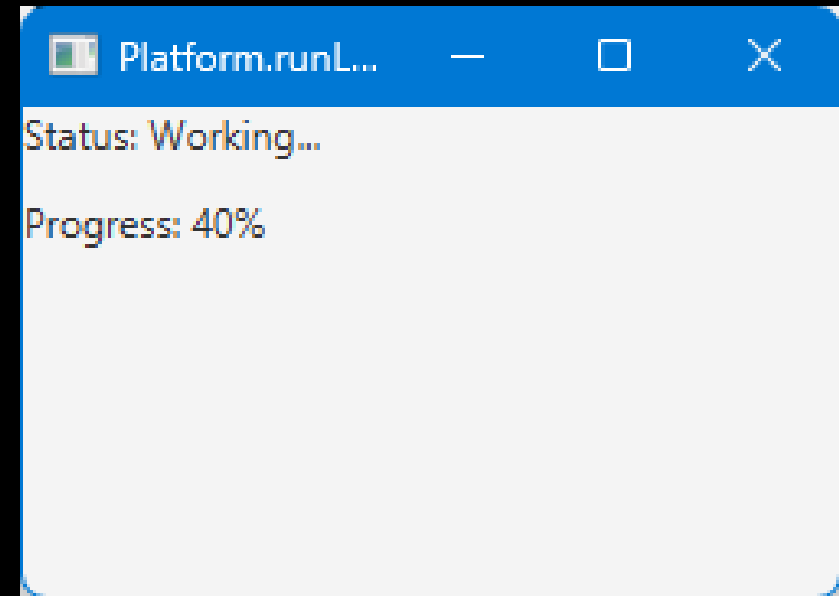
# Platform.runLater()

```
public class Gui extends Application {
    @Override
    public void start(final Stage stage) {
        Label statusLabel = new Label("Status: Waiting...");
        Label progressLabel = new Label("Progress: 0%");

        // Simulate background work with periodic UI updates
        new Thread() -> {
            for (int i = 1; i <= 10; i++) {
                try {
                    Thread.sleep(500); // Simulate work
                    int progress = i * 10; // Calculate progress
                    Platform.runLater() -> {
                        statusLabel.setText("Status: Working...");
                        progressLabel.setText("Progress: " + progress + "%");
                    };
                } catch (InterruptedException e) {
                    Platform.runLater() -> statusLabel.setText("Status: Interrupted!");
                    return;
                }
            }
        }
        // Final update on completion
        Platform.runLater() -> statusLabel.setText("Status: Completed!");
    }.start();

    VBox root = new VBox(10, statusLabel, progressLabel);
    stage.setScene(new Scene(root, 250, 150));
    stage.setTitle("Platform.runLater Enhanced Example");
    stage.show();
}

public static void main(final String[] args) {
    launch(args);
}
}
```





# CSS and FXML for JavaFX

- A. Style Sheets (CSS)
- JavaFX allows you to apply CSS to style your GUI components, enabling you to separate design from logic.

- 1. Create a styles.css file:
- ```
.label {  
    -fx-font-size: 16px;  
    -fx-text-fill: blue;  
}  
.button {  
    -fx-background-color: #FF6347; /* Tomato color */  
    -fx-text-fill: white;  
}
```

- 2. **\*\*Apply the CSS in JavaFX Code\*\***:

- ```
public class CSSExample extends Application {  
    @Override  
    public void start(final Stage stage) {  
        Label label = new Label("Styled Label");  
        Button button = new Button("Styled Button");  
  
        VBox root = new VBox(10, label, button);  
        Scene scene = new Scene(root, 300, 200);  
  
        // Load the CSS file  
        scene.getStylesheets().add(getClass().getResource("/styles.css").toExternalForm());  
  
        stage.setScene(scene);  
        stage.setTitle("CSS Example");  
        stage.show();  
    }  
  
    public static void main(final String[] args) {  
        launch(args);  
    }  
}
```

- B. FXML
- FXML is an XML-based language that allows you to define your GUI structure separately from the application logic.

- 1. Create an example.fxml file:
- ```
<?xml version="1.0" encoding="UTF-8"?>  
<?import javafx.scene.layout.VBox?>  
<?import javafx.scene.control.Label?>  
<?import javafx.scene.control.Button?>  
<VBox spacing="10" xmlns:fx="http://javafx.com/fxml">  
    <Label text="Hello from FXML!" />  
    <Button text="Click Me" />  
</VBox>
```

- 2. **\*Load FXML in Java Code**:
- ```
public class FXMLExample extends Application {  
    @Override  
    public void start(Stage stage) throws Exception {  
        // Load FXML file  
        Parent root = FXMLLoader.load(getClass().getResource("/example.fxml"));
```

- ```
        Scene scene = new Scene(root, 300, 200);  
        stage.setScene(scene);  
        stage.setTitle("FXML Example");  
        stage.show();  
    }  
}
```

- ```
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

- - CSS:
- - Use a \*.css file for styling.
- - Attach it to your Scene using scene.getStylesheets().add().

- - FXML:
- - Define UI in an XML file.
- - Load the FXML file using `FXMLLoader.load()`.

# CSS and FXML: Full Example

```
// styles.css
.label {
    -fx-font-size: 16px;
    -fx-text-fill: blue;
}
.button {
    -fx-background-color: #4CAF50;
    -fx-text-fill: white;
}

// example.fxml
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.Button?>
<VBox spacing="10" xmlns:fx="http://javafx.com/fxml">
    <Label text="This is styled!" />
    <Button text="Click Me" />
</VBox>
```

```
// Main.java

public class Main extends Application {
    @Override
    public void start(final Stage stage) throws Exception {
        Parent root = FXMLLoader.load(getClass()
            .getResource("/example.fxml"));
        Scene scene = new Scene(root, 300, 200);
        scene.getStylesheets().add(getClass()
            .getResource("/styles.css").toExternalForm());

        stage.setScene(scene);
        stage.setTitle("FXML and CSS Example");
        stage.show();
    }

    public static void main(final String[] args) {
        launch(args);
    }
}
```

**IMPORTANT:**

File

Project Structure...

Modules

Sources

Mark "resources" as "Resources"

2522 Lesson 10: GUI with JavaFX

